OPERATING SYSTEMS

PROJECT 1: SCHEDULING SIMULATION REPORT

SAUL GIL ID: 32239288

Introduction

Scheduling is a process into the operating systems where the system has to plan how to manage the resources and who to give it to, prioritazing the efficiency, basicly the main goals are resumed in asking these questions: what (what has to be done), who (who is going to do it) and when (the ideal fixed time for the what).

Main goals

The goal of this project is to create a code based in a round robin scheduling, the code must have the follow requierements:

Create 10 children processes ot from the parent process to run its time slices when they can.

Create a queue, this queue has to be managed by the parent process, it has to be like a referee to control who has access and who hasn't at certain time. In that queue, the child processes have to be divided in two queues, one for the ready state and the other for the wait queue, meaning that these children processes are not ready for the usage yet.

Create messages to make the children processes to communicate between each other.

Determine an appropiate time quantum given for the round robin based in ticks, when the designated quantum finishes, we have to move it to the queue. We have to manage the remaining time of the quantum for each child.

To put an alarm to register the time events.

For the child process, it has to receive the time slices to create the neccesary space for it, once it's done, it is going to simulate the ejecution with it's corresponding CPU burst and I/O Burst. This is going to be managed by creating an infinite loop.

Requirements

Visual studio code for the coding.

Wsl console to compile the code in visual studio once that it's done. This can be done with downloading Ubuntu for Windows/Mac (depending of your device).

Enough space in the hard-disk to receive the folder with information (About 1 MB).

How to run the code

The code is going to be run in visual studio code with the wsl console(or Linux/Ubuntu if prefered) executing the following commands:

gcc -o (output file name you want) scheduler.c (Make sure that you are in the same directory)

./(given name)

The last command is going to make you wait 1 minute to receive the data info for a minute of running the code, the document (schedule_dump.txt) is going to be placed in the current directory.

Associated files

The report is only a part of the assignment, the other files associated are the scheduler.c that is going to be copiled and executed, and the schedule_dump.txt that is going to give information of what is happening at the designated events.

How the code Works

The explanation of the code is the following (excluiding includes and structs, enums...):

Creation of a tick_handler, it creates a periodic clock tick.

Runq_...: consist of the creation a queue where the child ready are going to stay.

Wq_entry: wait queue.

In the main we first set the duration of the tick intervals, the time quantum, the max duration (in case we want to extend it more that the minute) and the file that is going to be written.

The we register the tick_handler to run on every tick and we créate a timer to trigger the alarm each tick.

Then we reach a key part of the code, the creation of the child, we do it with a loop with ends when we reach the value of MAX_CHILD, creating with fork all the children processes.

The next part of the code is the one where the children process are initializes the child process with an random CPU Burst and a random I/O burst, booting and sending it to the running queue.

The next while is where a process is created id the cpu is doing nothing and that process is sent to the running queue, then for the process that it's into I/O, the time remaining is reduced by one tick.

Continuing in the while loop, when the child is finished, it sends a message with msgrcv to notify and the parent moves the child to the wait queue, changing its state to wait.

Moving to the if (quantum[current..]), if the quantum is runned out, it preempt the process, setting it at the end of the queue, and then the CPU chooses the next process.

Finally the last part is about creating the output file with the information and killing all the children, and cleaning duties.

Output

Once that the compilation is correct, the following message should be shown:

```
saulgil@DESKTOP-3QQ48BQ:/mnt/c/Users/usuario$ cd /mnt/c/Users/usuario/Documents
saulgil@DESKTOP-3QQ48BQ:/mnt/c/Users/usuario/Documents$ gcc -o scheduler scheduler.c
saulgil@DESKTOP-3QQ48BQ:/mnt/c/Users/usuario/Documents$ ./scheduler
Done! Check schedule_dump.txt
saulgil@DESKTOP-3QQ48BQ:/mnt/c/Users/usuario/Documents$
```

Note that, as I have said before, when the ./scheduler is executed, nothing is going to be shown for a while because we have to get the data, wait patiently.

The file output is going to give the a big text that a part of the text should look like this:

(at time 0, process 507 gets cpu time, remaining cpu-burst 51) run-queue: [508 509 510 511 512 513 514 515 516] wait-queue: []

Once that the first quantum is done (50 ticks), the process should go to the back of the queue:

(at time 50, process 508 gets cpu time, remaining cpu-burst 183) run-queue: [509 510 511 512 513 514 515 516 507] wait-queue: [].